

Awesome Feature Navigation

Построение 2D-траектории робота по видео правой камеры и IMU

Проектная презентация

17 мая 2026 г.

- `awesome-feature-navigation` строит 2D-траекторию робота по офлайн-записи.
- Основные источники данных: видео правой камеры ZED, IMU CSV, timestamps кадров и Kalibr YAML-калибровки.
- Результат сохраняется как CSV и интерактивный HTML-график Plotly.
- Проект не заявляет полноценный visual-inertial SLAM: это прикладной пайплайн для текущей задачи и текущих данных.

Файл	Роль в пайплайне
data/ Right_cam.mp4	Видео правой камеры, где видна линия движения.
data/ imu_data.csv	Ускорения и угловые скорости IMU.
data/ timestamps2.csv	Реальные timestamps кадров из SVO/ZED.
data/imu- imu_calibration.yaml	Шумы IMU и random walk из Kalibr.
data/camchain- imucam-imu_ calibration.yaml	T_cam_imu, timeshift_cam_imu и параметры камеры.

Типовой запуск

Основная команда проекта:

```
uv run afn-run \  
  --video data/Right_cam.mp4 \  
  --imu data/imu_data.csv \  
  --config configs/right_camera.yaml \  
  --out outputs/right_trajectory
```

При необходимости добавляются `--save-debug`, `--save-loop-debug`, `--lap-bounds` и `--mode`.

Модуль	Ответственность
<code>cli.py</code>	CLI, конфиг, запуск пайплайна.
<code>calibration.py</code>	Timestamps и Kalibr YAML.
<code>imu_io.py</code>	IMU CSV, time shift, remap осей, поворот в camera frame.
<code>imu_preintegration.py</code>	IMU preintegration и fallback-интегратор.
<code>line_detection.py</code>	Детекция цветной линии и centerline.
<code>auto_config.py</code>	Автонастройка HSV и цвета линии.
<code>trajectory.py</code>	Оценка траектории, режимы, loop averaging.
<code>plotting.py</code>	CSV и HTML-графики.

Активный профиль лежит в `configs/right_camera.yaml`.

- `trajectory_mode: auto`: режим выбирается автоматически.
- `auto_prefer_tape_line: true`: сначала пробуется движение по линии.
- `target_color: blue`: для текущей записи линия синяя.
- `offline_tape_smoothing: true`: наблюдения линии сглаживаются по всей записи.
- `offline_adaptive_confidence: true`: порог надежности зависит от распределения `confidence`.
- `loop_strategy: representative_lap`: финал может быть одним замкнутым `representative lap`.

- Timestamps кадров и IMU переводятся из наносекунд в секунды.
- При абсолютном времени оба потока нормализуются относительно первого видеокadra.
- Kalibr-соглашение: $t_{imu} = t_{cam} + timeshift_{cam_imu}$.
- Для нарезки IMU по времени камеры применяется сдвиг $-timeshift_{cam_imu}$.
- Векторы IMU поворачиваются в camera frame через T_{cam_imu} .
- Для 2D-задачи включены gravity alignment и yaw-only подготовка.

Выбор режима траектории

- `tape_line`: основной режим для текущего видео, где видна синяя линия.
- `generic_vio`: fallback по optical flow и IMU yaw, если линия недостаточно надежна.
- `auto`: проверяет пригодность результата по числу точек, доле надежных кадров и spatial span.
- Monocular `generic_vio` дает относительный масштаб; метрический масштаб в `tape_line` задает `forward_speed_mps`.

- Кадр уменьшается, переводится в HSV, затем по blue HSV-диапазону строится маска линии.
- Из маски оставляется крупнейшая компонента, рабочая ROI очищается от нерелевантных областей.
- Centerline строится через distance transform, затем сглаживается.
- Для каждого кадра сохраняются `angle_rad`, `bottom_x`, `confidence` и IMU delta yaw.
- После офлайн-сглаживания интегрируются `x`, `y`, `yaw` с учетом `yaw correction` и `lateral correction`.

- На grayscale-кадрах выбираются feature points через goodFeaturesToTrack.
- Между кадрами точки отслеживаются Lucas-Kanade optical flow.
- Forward-backward check отбрасывает нестабильные треки.
- estimateAffinePartial2D оценивает 2D-transform с RANSAC.
- Visual yaw смешивается с IMU yaw; при слабом tracking вес IMU повышается.

Loop closure и финальный круг

- Для повторяющейся траектории проект пытается оценить период круга по compact grayscale descriptors.
- Полный путь сохраняется как `smoothed_traj`.
- Если круги согласованы по quality gate, строится замкнутый `final_traj`.
- При `representative_lap` финальная петля берется из лучшего наблюдаемого круга.
- Плохие круги отбрасываются по alignment RMSE и projection RMSE.

- `outputs/right_trajectory.csv`: финальная траектория t, x, y, yaw .
- `outputs/right_trajectory.html`: интерактивный Plotly-график.
- `*_raw.csv/html`: путь до offline-сглаживания.
- `*_smoothed.csv/html`: полный сглаженный многокруговой путь.
- `*_diagnostics.csv/html`: confidence, углы линии, скорость и IMU yaw delta.
- `*_laps.csv/html`: диагностика кругов и alignment.

- `pytest`: тесты калибровки, IMU IO, line detection и offline tape-line логики.
- `ruff`: стиль, импорты, аннотации и часть потенциальных ошибок.
- `myru`: статическая проверка типов для `src` и `tests`.
- `compileall`: проверка импортируемого Python-кода на синтаксис.
- Текущий coverage badge в репозитории показывает 34.8%.

Ограничения и допущения

- Видео `Right_cam.mp4` может не храниться в Git из-за размера.
- `tape_line` зависит от видимости синей линии и корректных HSV-настроек.
- `forward_speed_mps` задает метрический масштаб, поэтому ошибка скорости влияет на масштаб траектории.
- `generic_vio` без `depth` или `stereo` не восстанавливает надежный абсолютный масштаб.
- `Camera-IMU translation` загружается, но в текущей 2D-оценке позы не используется.

- Проект объединяет видео, IMU и Kalibr-калибровки в воспроизводимый CLI-пайплайн.
- Основной проектный сценарий - офлайн-навигация по синей линии с IMU yaw.
- Система сохраняет финальный результат, отладочные траектории и диагностику качества.
- Автоматический выбор режима и loop quality gate уменьшают риск принять плохую траекторию за финальную.